

Customer Churn Prediction & LTV Engine

Week 3 • Day 5

FastAPI — Batch Prediction Endpoint

Process thousands of customers at once with the POST `/predict/batch` endpoint

■ Day Objectives

- ◆ Build the POST `/predict/batch` endpoint for bulk predictions
- ◆ Accept CSV file uploads via the API
- ◆ Process predictions efficiently using vectorised operations
- ◆ Return results as a downloadable CSV or JSON
- ◆ Add rate limiting and logging middleware

■ 1. Batch vs Single Prediction

A **single prediction** endpoint (`/predict`) handles one customer at a time — useful for real-time scoring when a customer calls support. A **batch prediction** endpoint (`/predict/batch`) processes thousands of customers at once — used for nightly campaigns: 'Score all 7,000 customers and email the top-100 at-risk ones.'

■ 2. Batch Endpoint

■ Add `/predict/batch` to `src/api/main.py`

```

from fastapi import File, UploadFile
from fastapi.responses import StreamingResponse
import io

class BatchCustomer(BaseModel):
    customers: List[CustomerInput]

@app.post('/predict/batch')
async def predict_batch(batch: BatchCustomer):
    """
    Predict churn for a list of customers.
    Returns a list of predictions with risk levels and LTV.
    """
    results = []
    for i, customer in enumerate(batch.customers):
        # Reuse single prediction logic
        pred = await predict_churn(customer)
        results.append({
            'index': i,
            'churn_probability': pred.churn_probability,
            'churn_prediction': pred.churn_prediction,
            'risk_level': pred.risk_level,
            'predictive_ltv': pred.predictive_ltv,
            'ltv_segment': pred.ltv_segment,
        })
    return {
        'total_customers': len(results),
        'high_risk_count': sum(1 for r in results if r['risk_level']=='High'),
        'predictions': results
    }

@app.post('/predict/csv')
async def predict_from_csv(file: UploadFile = File(...)):
    """Accept a CSV file, return predictions as CSV."""
    content = await file.read()
    df = pd.read_csv(io.StringIO(content.decode('utf-8')))
    # ... same preprocessing as /predict ...
    # Return CSV response
    output = io.StringIO()
    df.to_csv(output, index=False)
    output.seek(0)
    return StreamingResponse(
        iter([output.getvalue()]),
        media_type='text/csv',
        headers={'Content-Disposition': 'attachment; filename=predictions.csv'}
    )

```

■ Test batch endpoint

```

curl -X POST http://localhost:8000/predict/batch \
-H 'Content-Type: application/json' \
-d '{"customers": [
  {"gender": "Male", "SeniorCitizen": 0, "Partner": "No", "Dependents": "No",
   "tenure": 2, "PhoneService": "Yes", "MultipleLines": "No",
   "InternetService": "Fiber optic", "OnlineSecurity": "No",
   "OnlineBackup": "No", "DeviceProtection": "No", "TechSupport": "No",

```

```
"StreamingTV":"No","StreamingMovies":"No",  
"Contract":"Month-to-month","PaperlessBilling":"Yes",  
"PaymentMethod":"Electronic check",  
"MonthlyCharges":70.35,"TotalCharges":140.70}  
]]'
```

■ 3. Commit & Checklist

- POST /predict/batch returns predictions for multiple customers
- high_risk_count included in response
- POST /predict/csv accepts file upload

■ Week3-Day5 Commit

```
git add .  
git commit -m "Week3-Day5: Batch prediction endpoint, CSV upload, streaming response"  
git push origin main
```

■ Estimated Time: 5 – 7 hours

— End of Week 3 Day 5 — Zaalima Development Confidential —